CS50, Vid8

Debugging
The "print" function itself is a good way of visualizing code in the terminal. The programmer can see the side effects that are present and learn how to deal with it.

Later on, we need a better method than print, one that does not disrupt the chronology of certain parts of our code.

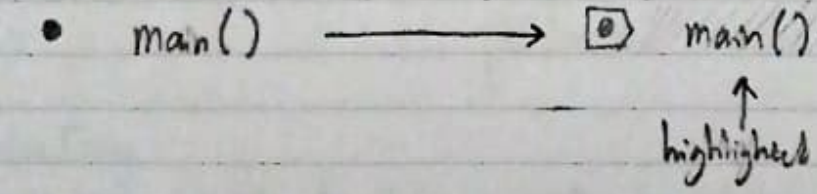
In many programming platforms, there is a debugging functionality in an activity bar.

breakpoint
- a mechanism that allows the user to specify on what line(s) of code they want to pause/break the execution of the program
- used in certain text editors or IDEs

☆ In VSCode, there are small opaque red dots that lie next to the line number. Clicking on that indicates to VSCode that I want it to break / pause the execution of my code just before something on that line happens.

☆ VSCode also has a small play button with a bug called "Run and Debug" which opens a panel with another button called "Run and Debug"

ex. Run and Debug is pressed



The line that is highlighted is where the breakpoint has been set, and the highlight itself means the function has not been executed yet, but will be executed when ready.

The "Run and Debug" option displays new icons that are attributed to the "Run and Debug" functionality.

▷▷ : Continue button - the break point will be left behind and the code is just going to continue to the end of the program

↷ : Step over - executes the highlighted line, but that is it. It will not step into the function (the function will not be called)

↓ : Step into - Step into the highlighted line, and then walk through the code step by step

↑ : Step back - almost the opposite of Step into; go backwards through the code

↻ reset

□ Stop

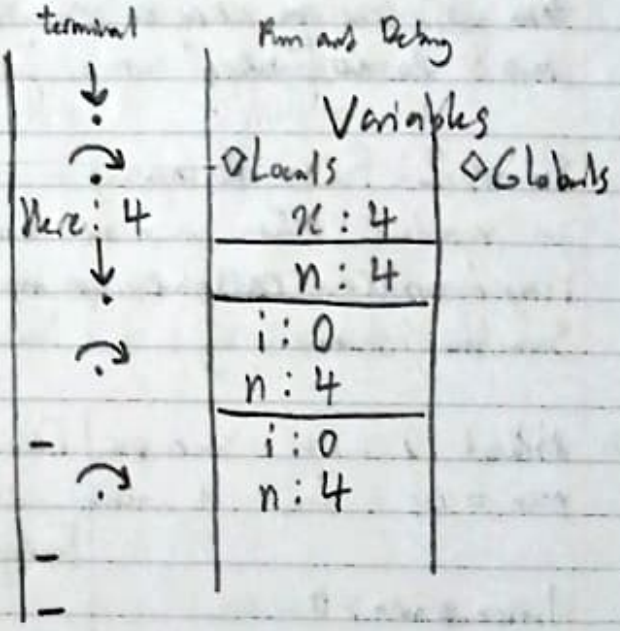
Under the "Run and Debug" panel, there is a mention of local and global variables.

ex.

```
def main():
    x = int(input("Here: "))
    poop(x)

def poop(n)
    for i in range(n):
        print("#" * i)

main()
```



Shorts 1: Handling Exceptions

When encountering errors, backtrack from the line where the error occurs to find the source/reason for the error. Also, the terminal can identify what type of error occurred.

Sometimes, when the datatype retrieved/using is not compatible with what we want to do with it, we can use the "try" and "except" blocks in an attempt to cast and convert the datatype into one that is usable, or return a statement that tells the user it is unable to be converted.

The user can specify what to do when specific errors occur by first identifying what type of error would ever occur. This is considered more optimal than just catching every error in general, because it helps programmers identify the error better.

Shorts 2: Raising Exceptions

Programmers can raise exceptions (intentional errors) to indicate something that is wrong in our human reasoning, but coded correctly.

ex.
```
miles = 10
min = 0

if not min > 0:
        raise Exception()
```

```
Error: ~~~~ line 5:
    raise Exception()
Exception
```

We as humans cannot run 10 miles in 0 minutes, so this reasoning raises an error to the user.

The user can specify what error has been raised. The previous example displayed a ValueError, so the user can specify:

```
if not min > 0:
    raise ValueError("Invalid Value bozo.")
```

So if the error occurs, the user would see:

```
Error: ~~~~ line 5:
    raise ValueError("Invalid Value bozo.")
ValueError: Invalid Value bozo.
```

There are certain conventions and cultures in the programming world where some people may say to specify and tell the user what the error is, while others may say to tell the user what to change.